

```
1 ### md
2 # IMT 575 Lab 2.17 - Steve Gonzales
3 ## Introduction
4
5 In this lab, you will design and implement MapReduce
  algorithms for a variety of common data processing tasks.
6
7 The MapReduce programming model (and a corresponding system)
  was proposed in a 2004 paper from a team at Google as a
  simpler abstraction for processing very large datasets in
  parallel. The goal of this assignment is to give you
  experience “thinking in MapReduce.” We will use small datasets
  that you can inspect directly to determine the correctness of
  your results and to internalize how MapReduce works.
8
9 You may want to review the material in module 2.16 to make
  sure you understand the programming model.
```

```
10 ### md
11 ## Setup
12
13 Download the data.zip file we will use in this assignment from
  Canvas.
14
15 To keep things simple, place all files in the same directory
  as this notebook.
```

```
16 ### md
17 ## A Simplified MapReduce Framework
18
19 Our goal is to practice thinking in terms of mapreduce without
  grappling with the systems issues associated with installing
  and configuring a complete distributed MapReduce environment.
20
21 To that end, we're providing you a simple Python MapReduce
  framework that returns results like the real thing, but does
  not involve parallel processing. We're writing programs that
  could be parallelized using MapReduce, but instead we're going
  to run them locally in Python.
22
23 MapReduce is a framework, not a programming language. The
  concepts can be used in any programming language. We're using
```

```
23 Python here for convenience.
24
25 The next cell provides a class definition that implements a
    simple MapReduce framework.
26 ###
27 import json
28
29 ###
30 ###
31 list_ex = []
32 type(list_ex)
33 tuple_ex = ()
34 type(tuple_ex)
35
36 print("List object implements: ", ' '.join([m for m in dir(
    list_ex) if not m.startswith('_')]))
37 print("Tuple object implements: ", ' '.join([m for m in dir(
    tuple_ex) if not m.startswith('_')]))
38
39 dict_ex = {}
40 type(dict_ex)
41 print("Dictionary object implements: ", ' '.join([m for m in
    dir(dict_ex) if not m.startswith('_')]))
42 dict_ex['a'] = ['abc']
43 dict_ex
44 # set_ex = {1, 3, 'a'}
45 # type(set_ex)
46 ###
47 # Main class for MapReduce functionality
48 class MapReduce:
49     def __init__(self):
50         self.intermediate = {}
51         self.result = []
52
53     def emit_intermediate(self, key, value):
54         self.intermediate.setdefault(key, [])
55         self.intermediate[key].append(value)
56
57     def emit(self, value):
58         self.result.append(value)
```

```

59
60     def execute(self, data, mapper, reducer):
61         result = []
62         for line in data:
63             record = json.loads(line)
64             mapper(record)
65
66         for key in self.intermediate:
67             reducer(key, self.intermediate[key])
68
69         #jenc = json.JSONEncoder(encoding='latin-1')
70         jenc = json.JSONEncoder()
71         for item in self.result:
72             print(jenc.encode(item))
73             # Added result output to use to compare to known
answers
74             result.append(json.loads(jenc.encode(item)))
75     return result

```

```

76 ### md
77 ## Helper Functions
78 ###
79 # The answer keys are large and it's impossible to tell if you
have a correct output unless one does a row by row comparison
.
80 # This function compares two lists of lists and returns a
dictionary with the differences.
81 def compare_list(list1, list2):
82     """Compare two multidimensional lists and return True if
they are equal, False otherwise.
83     The lists could be out of order. Display only the lines
that are different.
84     """
85     # Create a dict from the second list for quick lookup
86     d = {}
87     d['1'] = {item[0]: item[1] for item in list1}
88     d['2'] = {item[0]: item[1] for item in list2}
89     result = {'not_in_list1': [], 'not_in_list2': [], '
different_keys': []}
90     for d_name in ['1', '2']:
91         other_d_name = '1' if d_name == '2' else '1'

```

```

92         for k, v in d[d_name].items():
93             if k not in d[other_d_name]:
94                 print(f"Key {k} not found in list{
other_d_name}.")
95                 result[f'not_in_list{other_d_name}'].append(k
)
96             elif sorted(d[other_d_name][k]) != sorted(v):
97                 print(f"Different values for key {k}: {v} vs
{d[other_d_name][k]}")
98                 result['different_keys'].append(k)
99     result = {'not_in_list1': sorted(result['not_in_list1'
]), 'not_in_list2': sorted(result['not_in_list2']), '
different_keys': sorted(result['different_keys'])}
100     return result
101
102 # The files are not JSON, but are a list of lists, but not
even enclosed in brackets nor do they have commas separating
elements.
103 # This function converts the hacked files to a Python list.
104 def convert_hack_to_list(hacked_file):
105     """Convert a hacked file to a list of dictionaries."""
106     result = []
107     with open(hacked_file, 'r', encoding='utf-8') as f:
108         for line in f:
109             line = line.strip()
110             if not line:
111                 continue
112             result.append(eval(line))
113     return result
114
115 # Mini Unit Test
116 answer_file = 'inverted_index.json'
117 answers = convert_hack_to_list(answer_file)
118 print(answers[0], type(answers[0]))
119 attempt = [["Rossmore", ["edgeworth-parents.txt"]]]
120 r = compare_list(attempt, answers)
121 print(r)
122 ### md
123 ## Example: Word Count
124

```

```
125 The next cell contains an example of using the class above.
126
127 In Part 0, set up the name of the file. Modify this if you
    downloaded the assignment data to a different location.
128
129 In Part 1, we create a MapReduce object that is used to pass
    data between the map function and the reduce function; you
    won't need to use this object directly.
130
131 In Part 2, the mapper function tokenizes each document and
    emits a key-value pair. The key is a word formatted as a
    string and the value is the integer 1 to indicate an
    occurrence of word.
132
133 In Part 3, the reducer function sums up the list of
    occurrence counts and emits a count for word. Since the
    mapper function emits the integer 1 for each word, each
    element in the list_of_values is the integer 1.
134
135 The list of occurrence counts is summed and a (word, total)
    tuple is emitted where word is a string and total is an
    integer.
136
137 When you run this cell, you should see the results printed to
    the screen: a sequence of pairs (word, count) for every word
    found in the dataset.
```

```
138 #%%
139 # Part 0
140 filename = "books.json" # modify this if needed
141
142 # Part 1
143 mr = MapReduce()
144
145 # Part 2
146 def mapper(record):
147     # key: document identifier
148     # value: document contents
149     key = record[0]
150     value = record[1]
151     words = value.split()
```

```
152     for w in words:
153         mr.emit_intermediate(w, 1)
154
155 # Part 3
156 def reducer(key, list_of_values):
157     # key: word
158     # value: list of occurrence counts
159     total = 0
160     for v in list_of_values:
161         total += v
162     mr.emit((key, total))
163
164 # Part 4
165 with open(filename) as inputdata:
166     result = mr.execute(inputdata, mapper, reducer)
167 print(len(result))
```

```
168 """ md
169
170 ## Problem 1: Inverted Index
171 <font color='red'>(6 points)</font>
172
173
174 Create an inverted index. Given a set of documents, an
    inverted index is a dictionary where each word is associated
    with a list of the document identifiers in which that word
    appears.
175
176
177 ### Mapper Input
178
179 The input is a two-element list: [document_id, text], where
    document_id is a string representing a document identifier
    and text is a string representing the text of the document.
    The document text may have words in upper or lower case and
    may contain punctuation. You should treat each token as if it
    was a valid word; that is, you can just use value.split() to
    tokenize the string.
180
181
182 ### Reducer Output
```

```
183
184 The output should be a (word, document ID list) tuple where
    word is a String and document ID list is a list of Strings.
185
186 The correct output can be found in inverted_index.json


---


187 ###
188 # Part 0
189 filename = "books.json"
190
191 # Part 1
192 mr = MapReduce()
193
194 # Part 2
195 def mapper(record):
196     # key: document identifier
197     # value: document contents
198     doc_id = record[0]
199     text = record[1]
200     for word in text.split():
201         mr.emit_intermediate(word, doc_id)
202
203 # Part 3
204 def reducer(key, list_of_values):
205     # key: word
206     # value: list of document ids containing word
207     unique_doc_ids = sorted(set(list_of_values))
208     mr.emit((key, unique_doc_ids))
209
210 # Part 4
211 with open(filename) as inputdata:
212     invert_res = mr.execute(inputdata, mapper, reducer)
213 # Check against answer file
214 answers = convert_hack_to_list('inverted_index.json')
215 print(compare_list(invert_res, answers))


---


216 ### md
217 ## Problem 2: Relational Join
218     <font color='red'>(6 points)</font>
219
220 Implement a relational join as a MapReduce query.
221
```

222 Consider the following query:

```
223     SELECT *
224     FROM Orders, LineItem
225     WHERE Order.order_id = LineItem.order_id
```

226

227 This query produces all pairs of records from a table Orders and a table LineItem such that the order_id of the roder matches the order_id of the LineItem. This *join* operation is ubiquitous in databases.

228

229 Your MapReduce query should produce a result similar to this SQL query would return when executed against an appropriate database.

230

231 You can consider the two input tables, Order and LineItem, as one big concatenated bag of records that will be processed by the Map function record by record. That is, instead of two files, each with a separate table, you are given one big file with both tables inside it. Each record is labeled with which table it came from. For example, here are two lines in records.json:

232

```
233     ["order", "2", "78002", "0", "46929.18", "1996-12-01", "1
-URGENT", "Clerk#000000880", "0", " foxes. pending accounts
at the pending, silent asymptot"]
```

```
234     ["line_item", "2", "106170", "1191", "1", "38", "44694.46
", "0.00", "0.05", "N", "0", "1997-01-28", "1997-01-14", "
1997-02-02", "TAKE BACK RETURN", "RAIL", "ven requests.
deposits breach a"]
```

235

236 The first element of each record indicates which table the record is stored in. The other elements are the attribute values for this record. This example shows a key difference between databases and MapReduce: there is no schema in most MapReduce systems! You can put in any kind of crazy data you wish, but the tradeoff is that the database can't help protect you from bad data -- without a schema, it can't tell what's good and what's bad.

237

238


```
239 ### Map Input
240
241 Each input record is a list of strings representing a tuple
    in the database. Each list element corresponds to a different
    attribute of the table
242
243 The first item (index 0) in each record is a string that
    identifies the table the record originates from. This field
    has two possible values:
244
245 "line_item" indicates that the record is a line item.
246 "order" indicates that the record is an order.
247
248 The second element (index 1) in each record is the order_id.
249
250 LineItem records have 17 attributes including the identifier
    string.
251
252 Order records have 10 elements including the identifier
    string.
253
254 ### Reduce Output
255
256 The output should be a joined record: a single list of length
    27 that contains the attributes from the order record
    followed by the fields from the line item record. Each list
    element should be a string.
257
258 The correct output can be found in join.json


---


259 #%%
260 # Part 0
261 filename = "records.json"
262
263 # Part 1
264 mr = MapReduce()
265
266 # Part 2
267 def mapper(record):
268     # key: None
269     # value: a complete record, either an order or a line
```

```

269 item
270     mr.emit_intermediate(record[1], record)
271
272 # Part 3
273 def reducer(key, list_of_values):
274     orders = [r for r in list_of_values if r[0] == "order"]
275     line_items = [r for r in list_of_values if r[0] == "
        line_item"]
276     for order in orders:
277         for line_item in line_items:
278             # Exclude the table name from both records
279             joined = order + line_item
280             mr.emit((joined))
281
282 # Part 4
283 with open(filename) as inputdata:
284     join_list = mr.execute(inputdata, mapper, reducer)
285 # Check against answer file
286 answers = convert_hack_to_list('join.json')
287 for row in join_list:
288     found = False
289     for answer in answers:
290         if row == answer:
291             found = True
292             break
293     if not found:
294         print(f"**Row {row} not found in answers.")


---


295 ### md
296 ## Problem 3: Counting Friends
297     <font color='red'>(6 points)</font>
298
299 Consider a simple social network dataset consisting of a set
    of key-value pairs (person, friend) representing a friend
    relationship between two people. Describe a MapReduce
    algorithm to count the number of friends for each person.
300
301
302
303 ### Map Input
304

```

```
305 Each input record is a pair (personA, personB) where personA
    is a string representing the name of a person and personB is
    a string representing the name of one of personA's friends.
    Note that it may or may not be the case that the personA is a
    friend of personB.
306
307
308
309
310 ### Reduce Output
311
312 The output should be a pair (person, friend_count) where
    person is a string and friend_count is an integer indicating
    the number of friends associated with person.
313
314 The correct output can be found in friend_count.json


---


315 ###
316 # Part 0
317 filename = "friends.json"
318
319 # Part 1
320 mr = MapReduce()
321
322 # Part 2
323 def mapper(record):
324     # key: personA
325     # value: personB
326     mr.emit_intermediate(record[0], record[1])
327
328 # Part 3
329 def reducer(key, list_of_values):
330     mr.emit((key, len(list_of_values)))
331
332 # Part 4
333 with open(filename) as inputdata:
334     friend_list = mr.execute(inputdata, mapper, reducer)
335 # Check against answer file
336 answers = convert_hack_to_list('friend_count.json')
337 for row in friend_list:
338     found = False
```

```

339     for answer in answers:
340         if row == answer:
341             found = True
342             break
343     if not found:
344         print(f"**Row {row} not found in answers.")


---


345 ###
346 # Problem 3 but count unique friends with a modified file
with duplicates
347 filename = "friends_dupe.json"
348
349 def reducer_unique(key, list_of_values):
350     unique_friends = set(list_of_values)
351     mr.emit((key, len(unique_friends)))
352
353 print("Counting non unique friends...")
354 mr = MapReduce()
355 with open(filename) as inputdata:
356     friend_list = mr.execute(inputdata, mapper, reducer)
357 print("")
358
359 print("Counting unique friends...")
360 mr = MapReduce()
361 with open(filename) as inputdata:
362     unique_friend_list = mr.execute(inputdata, mapper,
363                                     reducer_unique)


---


363 ### md
364 ## Problem 4: Aymmetric Followers
365 <font color='red'>(6 points)</font>
366
367 The relationship "friend" is often symmetric, meaning that if
    I am your friend, you are my friend. Implement a MapReduce
    algorithm to check whether this property holds. Generate a
    list of all non-symmetric friend relationships.
368
369 ### Map Input
370 Each input record is a two-element list [personA, personB]
    where personA is a string representing the name of a person
    and personB is a string representing the name of one of
    personA's friends. Note that it may or may not be the case

```

```
370 that the personA is a friend of personB.
371
372 ### Reduce Output
373 The output should be all pairs (friend, person) such that (
    person, friend) appears in the dataset but (friend, person)
    does not.
374
375 Correct output: asymmetric_friendships.json


---


376 ###
377 # Part 0
378 filename = "friends.json"
379
380 # Part 1
381 mr = MapReduce()
382
383 # Part 2
384 def mapper(record):
385     person, friend = record
386     mr.emit_intermediate((person, friend), 1)
387
388 # Part 3
389 def reducer(key, list_of_values):
390     # key: (person, friend)
391     # Check if the reverse pair exists in the intermediate
    dict
392     reverse_key = (key[1], key[0])
393     if reverse_key not in mr.intermediate:
394         mr.emit([key[1], key[0]])
395
396 # Part 4
397 with open(filename) as inputdata:
398     asym_friends = mr.execute(inputdata, mapper, reducer)
399 # Check against answer file
400 answers = convert_hack_to_list('asymmetric_friendships.json')
401 for row in asym_friends:
402     found = False
403     for answer in answers:
404         if row == answer:
405             found = True
406             break
```

```
407         if not found:
408             print(f"**Row {row} not found in answers.")


---


409 ### md
410 ## Problem 5: Genetic data
411     <font color='red'>(6 points)</font>
412
413     Consider a set of key-value pairs where each key is sequence
        id and each value is a string of nucleotides, e.g.,
        GCTTCCGAAATGCTCGAA....
414
415     Design a MapReduce algorithm to remove the last 10 characters
        from each string of nucleotides, then remove any duplicates
        generated.
416
417
418
419
420 ### Map Input
421
422     Each input record is a two-element list (sequence id,
        nucleotides) where sequence id is a string representing a
        unique identifier for the sequence and nucleotides is a
        string representing a sequence of nucleotides
423
424
425
426
427 ### Reduce Output
428
429     The output from the reduce function should be the unique
        trimmed nucleotide strings.
430
431
432     Correct output: unique_trims.json
433


---


434 ###
435 # Part 0
436 filename = "dna.json"
437
438 # Part 1
```

```
439 mr = MapReduce()
440
441 # Part 2
442 def mapper(record):
443     mr.emit_intermediate(record[0], record[1][:-10]) # Trim
         the last 10 characters from the nucleotide string
444
445 # Part 3
446 def reducer(key, list_of_values):
447     unique_nucleotides = set(list_of_values) # Use a set to
         remove duplicates
448     mr.emit((key, list(unique_nucleotides)))
449
450 # Part 4
451 with open(filename) as inputdata:
452     dna_list = mr.execute(inputdata, mapper, reducer)
453 # Check against answer file
454 answers = convert_hack_to_list('unique_trims.json')
455 for row in dna_list:
456     found = False
457     for answer in answers:
458         if row[1][0] == answer:
459             found = True
460             break
461     if not found:
462         print(f"**Row {row} not found in answers.")
```